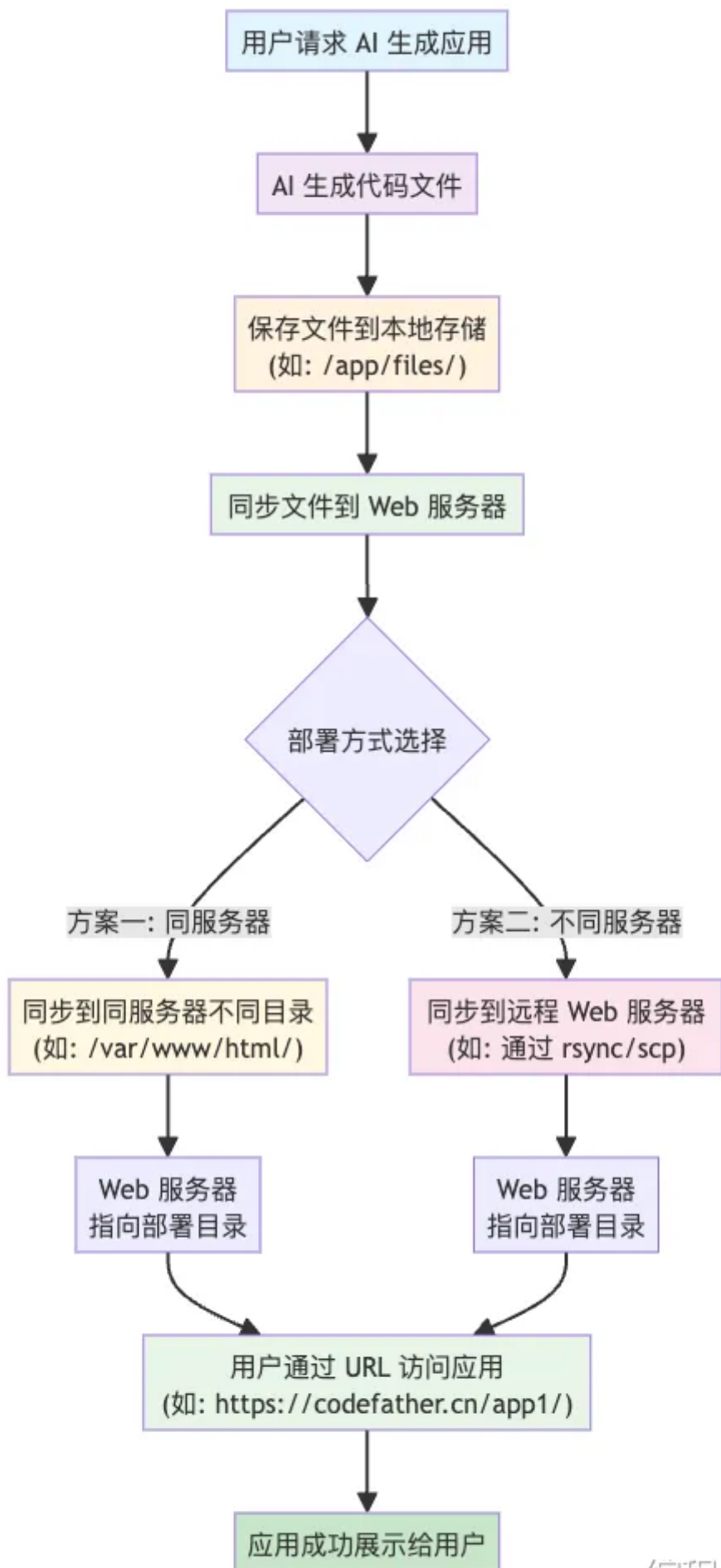


部署方案

部署的整体思路是：把本地生成的文件同步到一个 **Web 服务器** 上。可以是同一个服务器的不同目录，也可以是不同服务器，但显然前者成本更低。



我们有几种部署方案可以选择：

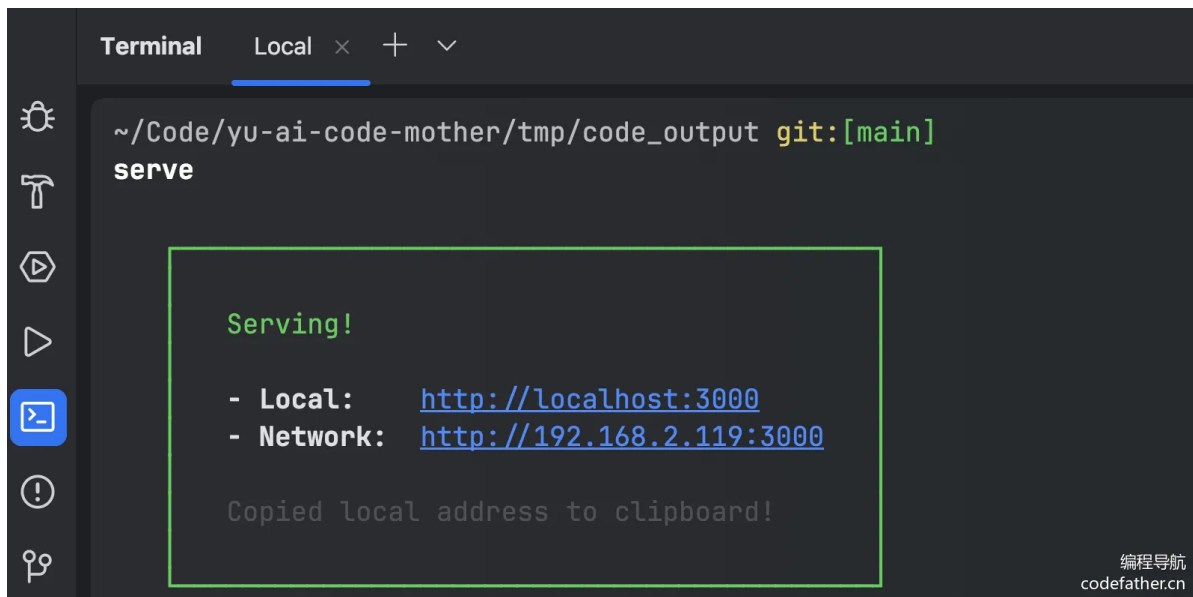
1、使用 Serve 工具

这是最简单的方案，通过 Node.js 的 `serve` 包可以快速启动一个 web 服务器，为指定目录提供 Web 访问服务。

先安装 `serve` 工具：

```
1 ▼java
2
3 复制代码npm i -g serve
```

假设 `code_output` 目录就是要部署的文件目录，只需要在这个目录内运行 `serve` 命令：

A terminal window titled 'Terminal' with tabs 'Local' and '+'. The current directory is '~/Code/yu-ai-code-mother/tmp/code_output' and the branch is 'git:[main]'. The command 'serve' has been executed. The output shows 'Serving!' followed by local and network addresses: 'Local: http://localhost:3000' and 'Network: http://192.168.2.119:3000'. A message 'Copied local address to clipboard!' is also shown. A green box highlights the output text. The terminal has a sidebar with icons for search, run and debug, extensions, play, a terminal icon (highlighted in blue), issues, and source control. A watermark '编程导航 codefather.cn' is in the bottom right corner.

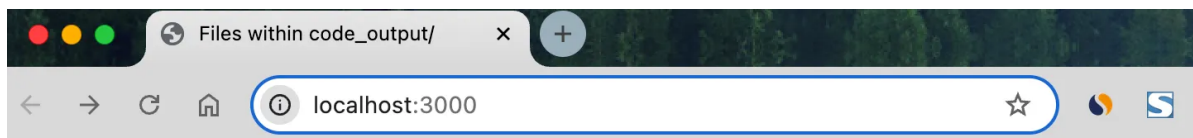
```
~/Code/yu-ai-code-mother/tmp/code_output git:[main]
serve

Serving!

- Local:    http://localhost:3000
- Network:  http://192.168.2.119:3000

Copied local address to clipboard!
```

就能查看到目录内的网站了：uS5zKidAFiYUU/5rUr3sN1c+0BwPgXRrvzOqOIs4XNo=



Index of `code_output/`

📁 `html_19451166587915386...` 📁 `multi_file_1945116048990...` 📁 `multi_file_19453737907115...`
📁 `multi_file_1945375928812...` 📁 `multi_file_1945376505789...`

编程导航
codefather.cn

访问对应的路径，直接就能访问到对应的网站：



使用时，只需提前在服务器上启动 serve 服务器，就能为特定部署目录提供 web 服务（比如 deployed），然后部署时将代码文件移动到这个目录下即可。

这种方案的优点是配置简单；缺点是依赖 Node.js 环境，需要独立启动 Web 服务进程，而且性能相对较低。

💡 当然，你也可以让 serve 服务器跟随 Spring Boot 项目启动或关闭，示例代码如下：

1) 使用命令行运行 serve：

```
1  ▼java复制代码@Service
2  public class ServeDeployService {
3
4      private static final String CODE_BASE_DIR = "/tmp/deploy";
5      private static final int SERVE_PORT = 3000;
6      private static Process serveProcess;
7
8      /**
9       * 启动 Serve 服务
10     */
11     public void startServeService() {
12         try {
13             if (serveProcess == null || !serveProcess.isAlive()) {
14                 ProcessBuilder pb = new ProcessBuilder(
15                     "npx", "serve", CODE_BASE_DIR, "-p",
16                     String.valueOf(SERVE_PORT)
17                 );
18                 pb.redirectErrorStream(true);
19                 serveProcess = pb.start();
20                 System.out.println("Serve service started on port " +
21                     SERVE_PORT);
22             }
23         } catch (Exception e) {
24             throw new RuntimeException("Failed to start serve service", e);
25         }
26     }
27
28     /**
29     * 关闭 Serve 服务
30     */
31     public void stopServeService() {
```

```

30         if (serveProcess != null && serveProcess.isAlive()) {
31             serveProcess.destroy();
32             try {
33                 serveProcess.waitFor(5, TimeUnit.SECONDS);
34                 System.out.println("Serve service stopped");
35             } catch (InterruptedException e) {
36                 serveProcess.destroyForcibly();
37                 System.out.println("Serve service force stopped");
38             }
39         }
40     }
41 }

```

2) 控制 serve 进程的生命周期:

```

1  ▼java复制代码@Component
2  public class ServeLifecycleManager {
3
4      @Autowired
5      private ServeDeployService serveDeployService;
6
7      /**
8       * Spring Boot 启动完成后启动 Serve 服务
9       */
10     @EventListener(ApplicationReadyEvent.class)
11     public void onApplicationReady() {
12         serveDeployService.startServeService();
13     }
14
15     /**
16      * Spring Boot 关闭时停止 Serve 服务
17      */
18     @PreDestroy
19     public void onApplicationShutdown() {
20         System.out.println("Shutting down Serve service...");
21         serveDeployService.stopServeService();
22     }
23 }

```

2、通过 Spring Boot 接口

我们可以直接在后端项目中实现一个静态资源服务接口，输入部署路径，返回相应的文件：

```

1  ▼java复制代码@RestController
2  @RequestMapping("/static")
3  public class StaticResourceController {
4
5      // 应用生成根目录（用于浏览）
6      private static final String PREVIEW_ROOT_DIR =
7      System.getProperty("user.dir") + "/tmp/code_output";
8
9      /**
10     * 提供静态资源访问，支持目录重定向
11     * 访问格式：http://localhost:8123/api/static/{deploykey}/{filename}]

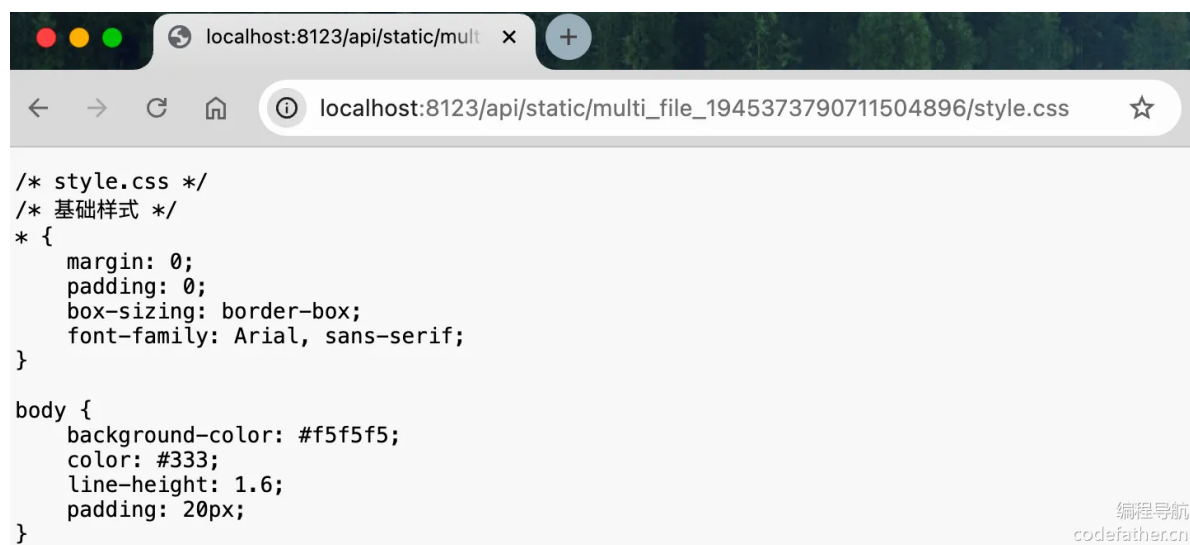
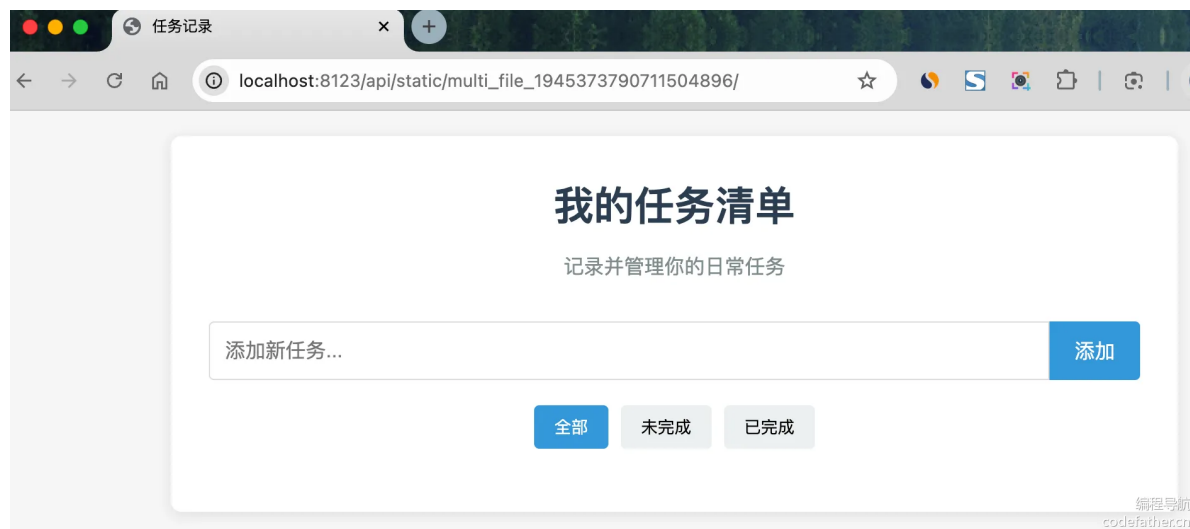
```

```

11     */
12     @GetMapping("/{deployKey}/**")
13     public ResponseEntity<Resource> serveStaticResource(
14         @PathVariable String deployKey,
15         HttpServletRequest request) {
16         try {
17             // 获取资源路径
18             String resourcePath = (String)
19 request.getAttribute(HandlerMapping.PATH_WITHIN_HANDLER_MAPPING_ATTRIBUTE);
20             resourcePath = resourcePath.substring("/static/" +
21 deployKey).length());
22             // 如果是目录访问（不带斜杠），重定向到带斜杠的URL
23             if (resourcePath.isEmpty()) {
24                 HttpHeaders headers = new HttpHeaders();
25                 headers.add("Location", request.getRequestURI() + "/");
26                 return new ResponseEntity<>(headers,
27 HttpStatus.MOVED_PERMANENTLY);
28             }
29             // 默认返回 index.html
30             if (resourcePath.equals("/")) {
31                 resourcePath = "/index.html";
32             }
33             // 构建文件路径
34             String filePath = PREVIEW_ROOT_DIR + "/" + deployKey +
35 resourcePath;
36             File file = new File(filePath);
37             // 检查文件是否存在
38             if (!file.exists()) {
39                 return ResponseEntity.notFound().build();
40             }
41             // 返回文件资源
42             Resource resource = new FileSystemResource(file);
43             return ResponseEntity.ok()
44                 .header("Content-Type",
45 getContentTypeWithCharset(filePath))
46                 .body(resource);
47         } catch (Exception e) {
48             return
49 ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
50         }
51     }
52
53     /**
54      * 根据文件扩展名返回带字符编码的 Content-Type
55      */
56     private String getContentTypeWithCharset(String filePath) {
57         if (filePath.endsWith(".html")) return "text/html; charset=UTF-8";
58         if (filePath.endsWith(".css")) return "text/css; charset=UTF-8";
59         if (filePath.endsWith(".js")) return "application/javascript;
60 charset=UTF-8";
61         if (filePath.endsWith(".png")) return "image/png";
62         if (filePath.endsWith(".jpg")) return "image/jpeg";
63         return "application/octet-stream";
64     }
65 }

```

测试访问，效果如图：



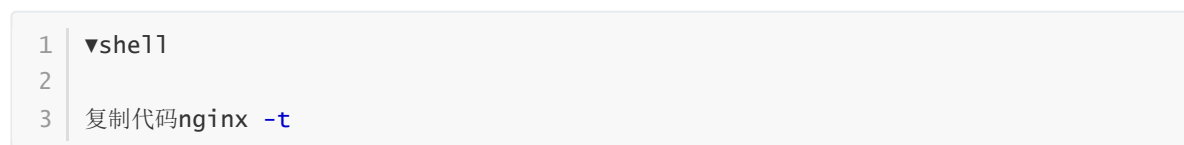
这种方案的优点是无需额外进程，非常方便；缺点是功能相对简单，性能也不如专业的 Web 服务器。

3、使用 Nginx 映射

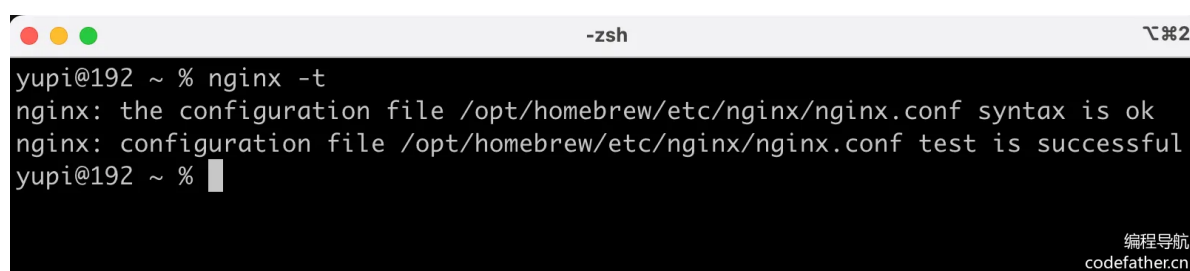
Nginx 是专业的 Web 服务器，性能优异，功能丰富。**因此这是最推荐的生产环境方案。**

1) 首先到 [Nginx 官网](#) 下载 Nginx，各个操作系统的安装包都有。

下载好 Nginx 后，找到 Nginx 配置文件 `nginx.conf`，Windows 系统直接到安装目录下找，Mac / Linux 系统可以通过命令查看 Nginx 配置文件位置：



如图：



2) 修改 Nginx 配置，http 块中添加 server 块，配置 root 为项目部署根目录：

```
1  ▼nginx复制代码# 静态资源服务器 - 80 端口
2  server {
3      listen      80;
4      server_name localhost;
5      charset      utf-8;
6      charset_types text/css application/javascript text/plain text/xml
application/json;
7      # 项目部署根目录
8      root          /Users/yupi/Code/you-ai-code-mother/tmp/code_deploy;
9
10     # 处理所有请求
11     location ~ ^/([^\/]+)/(.*)$ {
12         try_files /$1/$2 /$1/index.html =404;
13     }
14 }
```

注意，Windows 系统的路径斜杠要相反，比如 `C:/code/you-ai-code-mother_live/tmp/code_deploy`

上述配置中使用了 `try_files` 指令，能够按顺序尝试多个文件路径，从而更灵活地处理文件访问。举个例子，当访问 `/app/style.css` 时，会先尝试找到 `/app/style.css` 文件，如果不存在则返回 `/app/index.html`，最后才返回 404 错误，这样能够适配后续我们要生成的 Vue 单页面应用。

💡 `try_files` 指令的具体解释：

- `/$1/$2`：第一个尝试的路径
- `/$1/index.html`：第二个尝试的路径
- `=404`：如果都找不到，返回 404 错误

举些例子：

访问URL	\$1	\$2SqYALXd6fiZb5TX5KnIKAdatFLjFVUdQRN33LmleEms=	尝试路径1	尝试路径2
/app/	app	""	/app/	/app/index.html
/app/style.css	app	style.css	/app/style.css	/app/index.html
/app/js/main.js	app	js/main.js	/app/js/main.js	/app/index.html

3) 启动 Nginx，或者输入命令来重载配置：

```
1  ▼bash
2
3  复制代码nginx -s reload
```

然后直接访问 80 端口，就能看到网站了：



SqYALXd6fiZb5TX5KnIKAdatFLjFVUdQRN33LmleEms=

这种方案的性能最佳，最适合生产环境；缺点是需要额外引入 Nginx 组件。